



Circular Queue

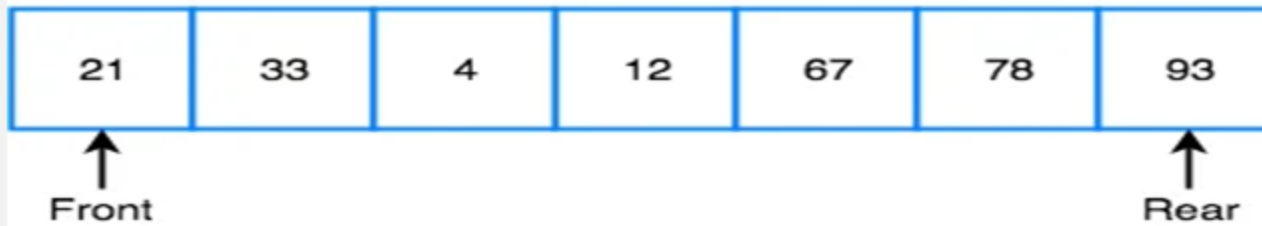
1. Concept Of Circular Queue
2. Circular Queue Data structure
3. Application of Circular Queue.
4. Working Of Circular Queue.

- Implementation of a linear queue brings the drawback of memory wastage. However, memory is a crucial resource that you should always protect by analyzing all the implications while designing algorithms or solutions. In the case of a linear queue, when the rear pointer reaches the MaxSize of a queue, there might be a possibility that after a certain number of dequeue() operations, it will create an empty space at the start of a queue.

1. **Memory Management in Operating System:** In operating systems, processes are stored in a circular process queue wherein the old processes are removed for execution and new processes are added or queued. The unused memory locations post process dequeue are utilized using circular queues which would have not been possible in ordinary queues.
2. **Traffic system:** In computer controlled traffic systems, circular queues are used to switch on the traffic lights one by one repeatedly as per the time set.
3. **CPU Scheduling:** Operating systems often maintain a queue of processes that are ready to execute or that are waiting for a particular event to occur.

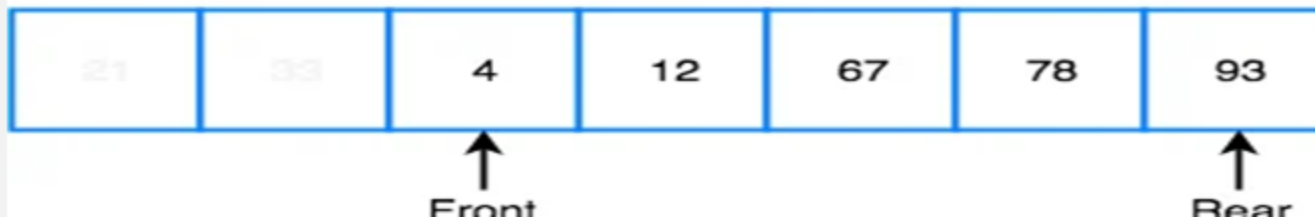
In a Linear queue, once the queue is completely full, it's not possible to insert more elements. Even if we dequeue the queue to remove some of the elements, until the queue is reset, no new elements can be inserted.

Queue is Full



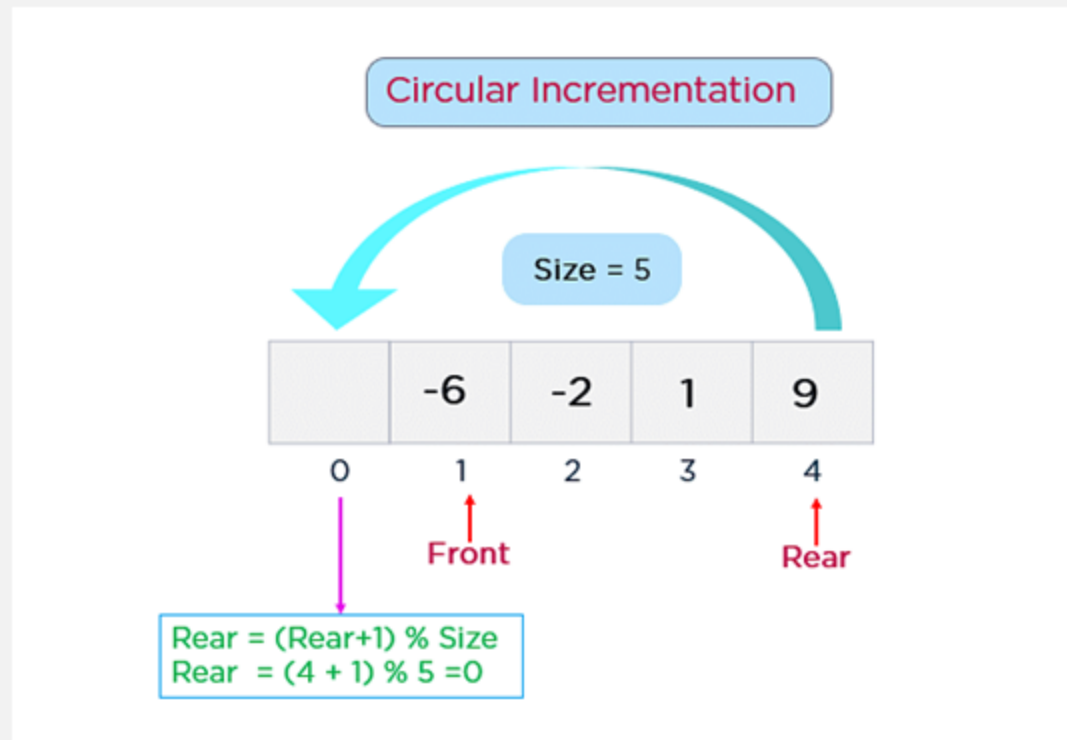
When we dequeue any element to remove it from the queue, we are actually moving the front of the queue forward, thereby reducing the overall size of the queue. And we cannot insert new elements, because the rear pointer is still at the end of the queue.

Queue is Full (Even after removing 2 elements)

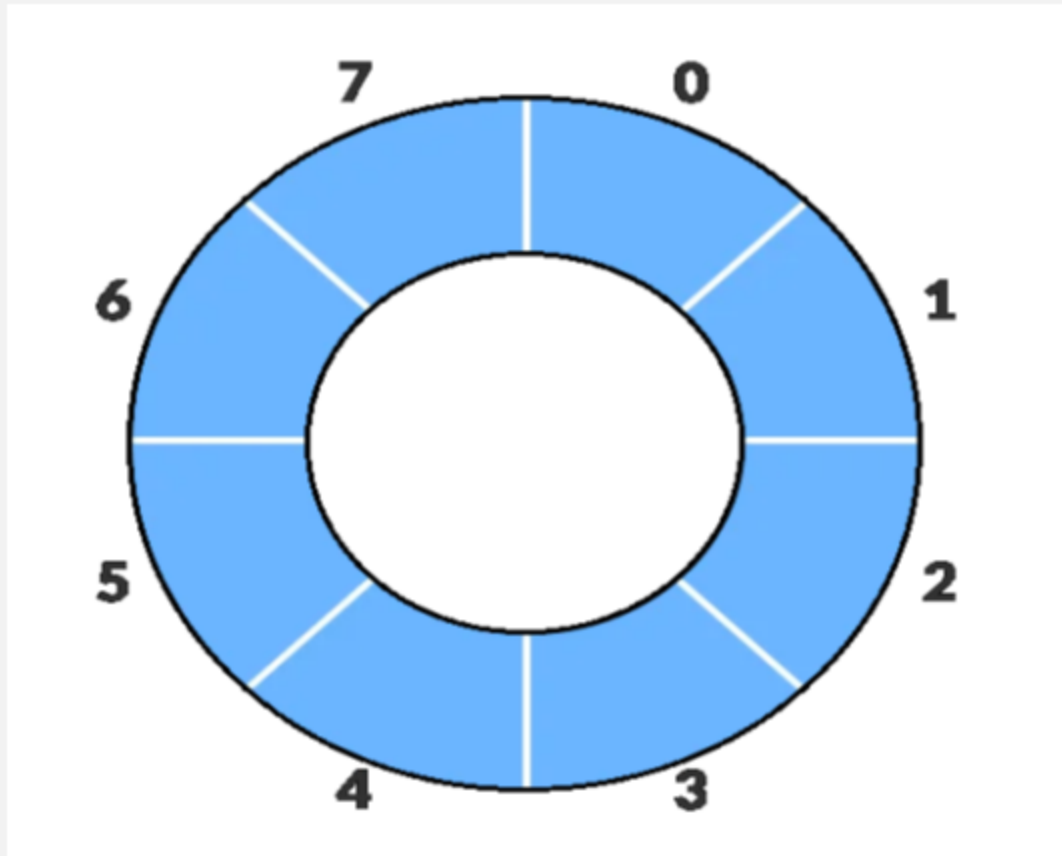


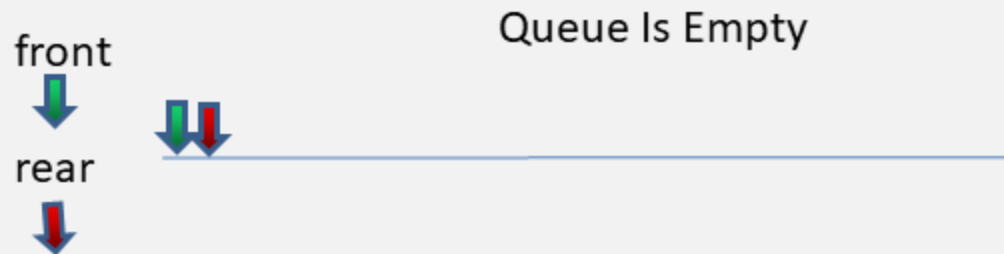
The MaxSize of your queue is 5, and the rear pointer has already reached the end of a queue. There is one empty space at the beginning of a queue, which means that the front pointer is pointing to location 1.

Instead of incrementing rear by 1, we should change rear by $\text{rear} = (\text{rear} + 1) \% \text{size}$



Working of Circular Queue.





-1 0 1 2 3 4

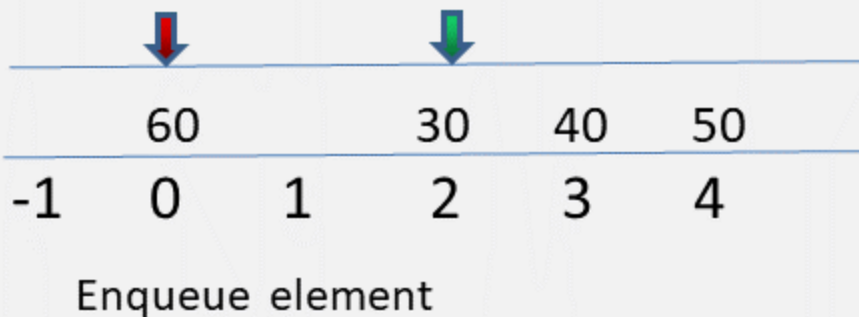
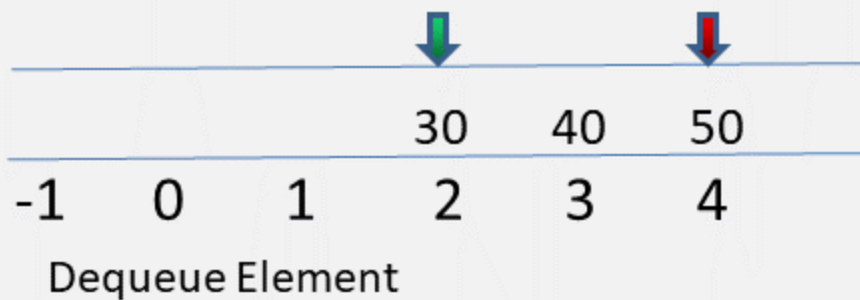
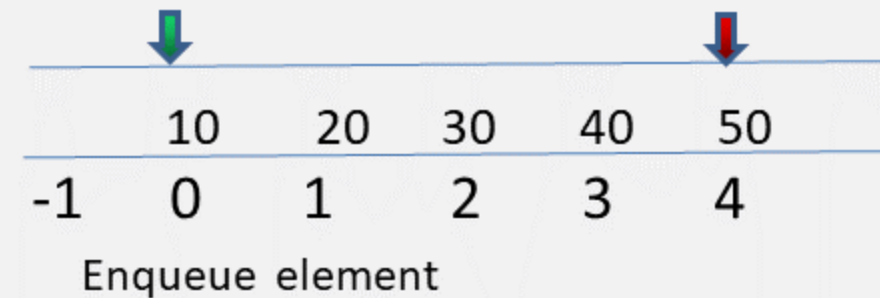


-1 0 1 2 3 4

Enqueue First element

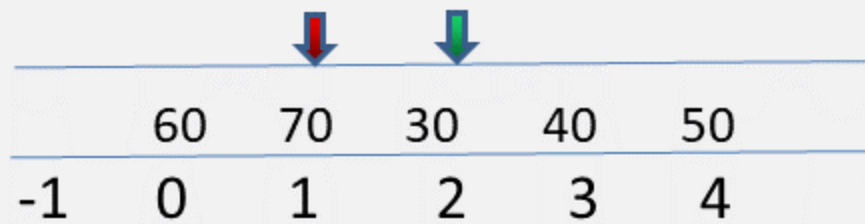


Enqueue Second element



Queue Full

1. $\text{front} == 0 \ \&\& \ \text{rear} == \text{size}-1$



A diagram of a circular queue with 5 slots. The slots are indexed from -1 to 4. The values in the slots are 60, 70, 30, 40, and 50 respectively. A red arrow points to the slot at index 1 (value 70), and a green arrow points to the slot at index 2 (value 30).

	60	70	30	40	50
-1	0	1	2	3	4

Queue Full

1. $\text{front} == (\text{rear} + 1) \% \text{size}$